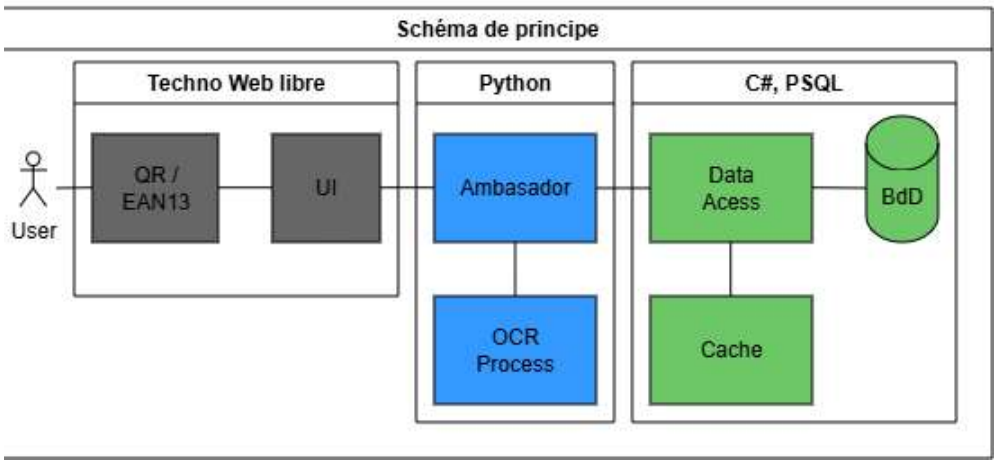
R5.A.7 Automatisation de la production d'application

Mise en pratique de la conteneurisation

Création d'une application de manipulation de QR / EAN13 avec une IHM web

Travail à réaliser en binôme - rendu vers la fin des cours du S5 (phase2)

Schéma de principe - point d'entrée HTTP : port 8080



Point important : le port 8080 est le point d'entrée unique vu depuis le poste utilisateur. Plusieurs conteneurs ne peuvent pas publier simultanément le même port 8080 de l'hôte. Les échanges entre conteneurs passent donc par le réseau Docker. L'accès externe à l'application doit se faire via http://localhost:8080.

La mise en situation est organisée en deux phases complémentaires :

1) analyse fonctionnelle et formalisation ;

2) transformation en architecture technique puis conteneurisation avec Docker.

Les livrables de la phase 1 servent d'entrée à la phase 2.

Définition des services

Service	Technologie	Responsabilités	Entrées / sorties principales
IHM Web	Framework web libre	Sélection ou acquisition d'un QR / EAN13 ; envoi de la demande ; affichage du résultat. L'IHM ne doit pas accéder directement à la base de données.	Entrée utilisateur / requête HTTP / résultat affiché
Ambassador / API	Python	Point d'orchestration logique : reçoit la requête, valide les paramètres, appelle les services nécessaires et renvoie une réponse structurée à l'IHM.	Requête API / JSON / codes de retour
OCR / Decode Process	Python	Extraction ou décodage QR / EAN13, filtrage et normalisation des données ; traitement métier nécessaire avant stockage ou restitution.	Image ou valeur brute / valeur décodée et normalisée
Data Access	C#	Accès unique à PostgreSQL : lecture, écriture, recherche et mise à jour. Aucun autre service ne doit accéder directement à PostgreSQL.	Requêtes de données / objets ou JSON / statut
Cache	C#/.NET ou technologie justifiée	Conserver temporairement des résultats utiles pour éviter des accès répétés. Son usage et sa stratégie d'invalidation doivent être explicités.	Clé / valeur / durée de vie
Base de données	PostgreSQL	Stockage persistant des données métier et des résultats utiles. Le schéma minimal est défini par le binôme en cohérence avec l'analyse.	Données persistantes

Règle d'architecture : les responsabilités doivent rester séparées. L'IHM gère la présentation, le Data Process traite les données et le Data Access gère la persistance. Le découpage en conteneurs doit refléter ce découplage, sans imposer qu'un bloc fonctionnel corresponde nécessairement à un seul conteneur : le choix doit être justifié.

Phase 1 - Analyse et formalisation (Bruno Marhic)

Livrable : R5A7_NOM_cdc.pdf (page de garde avec les noms du binôme).

Partie 1 - Analyse du besoin courte / méthode APTE

- 2 à 3 pages maximum.
- Verbalisation du besoin : à qui le système rend-il service, sur quoi agit-il, dans quel but ?
- Diagramme pieuvre.
- Tableau des services fonctionnels : fonctions principales (FP) et fonctions contraintes (FC), identifiées et priorisées par numéro.
- Les contraintes techniques imposées (Web, Python, C#, PostgreSQL, Docker, point d'entrée 8080) doivent être distinguées des fonctions de service.
- Exemple succinct de caractérisation d'une fonction de service :

Fonction de service	Critère d'appréciation	Niveau attendu	Flexibilité

- L'exemple ci-dessus illustre seulement la forme attendue du tableau. Chaque équipe doit identifier, formuler et caractériser ses propres fonctions de service.

Partie 2 - Analyse fonctionnelle de niveau 2

- A.H.F. : arbre hiérarchique fonctionnel, avec une décomposition en profondeur cohérente (AD).
- Pour les fonctions retenues jusqu'au niveau 2, produire les spécifications fonctionnelles :
    - ID, Nom, Valeur ajoutée, entrées/sorties, principe algorithmique ou logigramme (si nécessaire)

L'analyse fonctionnelle reste indépendante du choix des conteneurs et des ports internes.

Partie 3 - DFD N0 et N1 : flux de données fonctionnels

- Réaliser un DFD niveau 0 (contexte) puis un DFD niveau 1 selon la norme Yourdon.
- Nommer les flux par les données échangées : image QR/EAN13, valeur décodée, données normalisées, résultat, etc.
- Le DFD doit représenter le point de vue fonctionnel et non l'architecture technique Docker.

Partie 4 - Diagramme de séquence UML N1

- Décrire au minimum le scénario nominal complet :
    - Utilisateur → IHM → Backend/Ambassador → Data Process / Data Access → IHM.
- Utiliser des lignes de vie représentant des acteurs ou composants logiques, et non des langages.
- Les fragments combinés alt, opt et loop peuvent être utilisés lorsqu'ils sont pertinents :
    - erreur de décodage, résultat absent, traitement optionnel, répétition ou rafraîchissement.

Cohérence attendue : les fonctions de l'AHF, les processus du DFD et les interactions du diagramme de séquence doivent décrire le même système, sous trois points de vue différents : structure, flux de données et chronologie.

Phase 2 - Mise en pratique Docker (Nicolas Delpech)

Livrable documentaire :

- R5A7_NOM_code.zip (contenu de code du projet, sans les binaires ou tout autre librairie, executable, etc.).
- R5A7_NOM_dfd.pdf (DFD orienté technique)
- R5A7_NOM_doc.pdf (explication du fonctionnement, découpage fonctionnel, isolation et autre mécanisme implémenté)

Objectif : transformer les modèles fonctionnels de la phase 1 en une architecture technique reproductible, puis déployer l'application complète avec Docker Compose.

Partie 5 - Architecture technique et conteneurisation

- Reprendre le document le DFD N1 fonctionnel validé en phase 1.
- Produire un DFD N1-orienté technique Docker / schéma d'architecture technique :x don
  - processus fonctionnels → services techniques → conteneurs → API → stockages.
- Indiquer, pour chaque conteneur :
  - responsabilité, image de base, port interne, dépendances, variables d'environnement ...
- Tous les accès depuis le poste utilisateur doivent passer par le **port 8080**.
- L'isolation doit etre stricte au sein du container, ex: le front ne doit pas accéder à la base de données (possible d'utiliser plusieurs vnet, de mettre des stratégies, etc)

Étapes intermédiaires de réalisation - jalons conseillés

Jalon	Travail attendu	Critère de validation
J0 - Contrat d'interface	Fixer les échanges avant de conteneuriser : endpoints/API, méthode HTTP, paramètres, format JSON, codes de retour, données stockées. Définir également les noms des services Docker et les variables d'environnement nécessaires.	Un tableau d'API et un schéma technique cohérents.
J1 - Base PostgreSQL	Créer le schéma minimal et les données d'initialisation. Tester la création de la base et la persistance. Prévoir un volume Docker. Identifiants dédiés au data access	La base redémarre sans perdre les données utiles. Le data access utilise ses propres identifiants et pas le compte root de la bdd
J2 - Data Access C#	Implémenter et tester le service C# seul, puis le connecter à PostgreSQL. Ajouter un Dockerfile. Vérifier lecture/écriture par curl, navigateur ou client HTTP.	Le service répond et accède correctement à PostgreSQL. Implémentation d'un swagger
J3 - Data Process Python	Implémenter le traitement QR/EAN13 et les opérations de filtrage/normalisation. Conteneuriser. Tester d'abord avec des données de test, puis via l'API prévue.	Le service produit un résultat déterministe à partir d'une entrée connue.
J4 - Communication inter-services	Relier Data Process/Ambassador et Data Access sur un réseau Docker. Les services se joignent par leur nom Docker. Gérer les erreurs et les réponses non valides.	Un appel métier complet circule entre au moins deux services conteneurisés.
J5 - IHM Web	Conteneuriser l'IHM. Connecter l'entrée QR/EAN13, le déclenchement du traitement et l'affichage du résultat. L'IHM ne communique pas directement avec PostgreSQL.	Le scénario utilisateur fonctionne de bout en bout.
J6 - Docker Compose	Assembler tous les conteneurs dans docker-compose.yml : réseaux, volumes, variables d'environnement, dépendances et healthchecks utiles. Mettre en place le point d'entrée unique 8080.	docker compose up --build démarre l'ensemble depuis un poste propre. Utiliser des images officiels en dehors des images de build Les Dockerfile doivent être propre et produire des images propres, saines et LEGERES (ne contient que le strict minimum)
J7 - Validation et robustesse	Tester arrêt/redémarrage, persistance, données invalides, service indisponible et logs. Vérifier qu'aucun secret n'est codé en dur et qu'aucun service interne n'est inutilement publié sur l'hôte. Implémenter des TU sur l'ensemble du code produit Un script situé à la racine du projet doit permettre d'executer les TU sur les différents projet	Scénarios de test documentés et comportement explicite en cas d'erreur.
J8 - Livraison reproductible	Nettoyer le dépôt, compléter le README, supprimer les dépendances générées et fournir les commandes de construction/lancement. Refaire un lancement complet à partir d'un clone ou d'une archive propre.	Une autre machine peut reconstruire et lancer le projet sans manipulation manuelle dans les conteneurs.

Organisation réseau recommandée

- Port exposé côté hôte : 8080 uniquement.
- Il est possible si utilisation d'une fullstack (front / back séparé ce qui correspond à un usage entreprise) de mettre en place un proxy (nfginx, caddy, etc.) pour répartir les flux. Seuls des flux web sont tolérés et dédiés à l'IHM (front + back).
- Des isolations via différents network sont mises en place (un hôte peut avoir plusieurs networks)

- Les URL inter-conteneurs utilisent les noms Docker, par exemple http://data-access:<port-interne> et non http://localhost:<port>.

Pourquoi cette règle ? Docker ne permet pas à plusieurs conteneurs de publier simultanément le même couple adresse/port de l'hôte. "Tout sur le port 8080" signifie donc : un point d'entrée public unique sur 8080, puis un routage interne entre services.

Fichiers à rendre (R5A7_NOM_code.zip) - phase Docker

Élément	Contenu attendu
docker-compose.yml	orchestration complète de l'application
Dockerfile(s)	un Dockerfile pour chaque image applicative construite par le binôme
.env.example	variables attendues sans mot de passe ou secret réel
README.md	prérequis, commandes de build/lancement/arrêt, URL de test et description rapide des services
Sources	répertoires IHM, C#, Python et éventuels scripts d'initialisation/migration PostgreSQL/proxy
Tests	script, commandes curl, collection HTTP ou tableau de tests permettant de vérifier les API principales

Critère de livraison principal : à partir du dossier racine du projet, la commande docker compose up --build doit permettre de reconstruire et démarrer l'application complète. L'application doit ensuite être accessible depuis le poste hôte via http://localhost:8080.